

1. Проверка установки ПО

2. Запуск RabbitMQ

Запуск контейнера:

```
PROBLEMS TERMINAL OUTPUT PORTS ... powershell + ~ ▢ ✕ ⋮ [ ]
```

```
PS C:\Users\Admin\Documents\weather_aggregator_system> docker-compose up -d
time="2025-10-31T17:07:21+03:00" level=warning msg="C:\Users\Admin\Documents\weather_aggregator_system\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
```

```
[+] Running 11/11
✔ rabbitmq Pulled                                12.6s
✔ ce334ee7a96e Pull complete                      8.5s
✔ 5dfddc3658b8 Pull complete                      7.4s
✔ 50095a1a3490 Pull complete                      8.7s
✔ 5a061f63f03b Pull complete                     1.0s
✔ 8ddbdcbe86b0 Pull complete                      0.9s
✔ a7620a19f6fc Pull complete                      0.8s
✔ 450217c3094e Pull complete                      9.4s
✔ e32118121bc1 Pull complete                     1.2s
✔ 1fecf0e475be Pull complete                      7.2s
✔ 4b3ffd8ccb52 Pull complete                      6.4s

[+] Running 3/3
✔ Network weather_aggregator_system_default      Created          0.1s
✔ Volume weather_aggregator_system_rabbitmq_data Created         0.0s
✔ Container weather-rabbitmq                      Started         1.3s
```

<http://localhost:15672>

RabbitMQ Management

localhost:15672

RabbitMQ™

Username: admin

Password:

Login

RabbitMQ™

RabbitMQ 3.13.7 Erlang 26.2.5.15

Refreshed 2025-10-31 15:09:43 Refresh every 5 seconds

Virtual host All

Cluster rabbit@f75a43d1e398

User admin Log out

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Totals

Queued messages last minute

Currently idle

Message rates last minute

Currently idle

Global counts

Connections: 0 Channels: 0 Exchanges: 7 Queues: 0 Consumers: 0

Nodes

Name	File descriptors	Socket descriptors	Erlang processes	Memory	Disk space	Uptime
rabbit@f75a43d1e398	44 1048576 available	0 943629 available	451 1048576 available	172 MiB 6.1 GiB high watermark	947 GiB 48 MiB low watermark	2m 1s

Churn statistics

Ports and contexts

Export definitions

Import definitions

HTTP API

Documentation

Tutorials

New releases

Commercial edition

Commercial support

Discussions

Discord

Plugins

GitHub

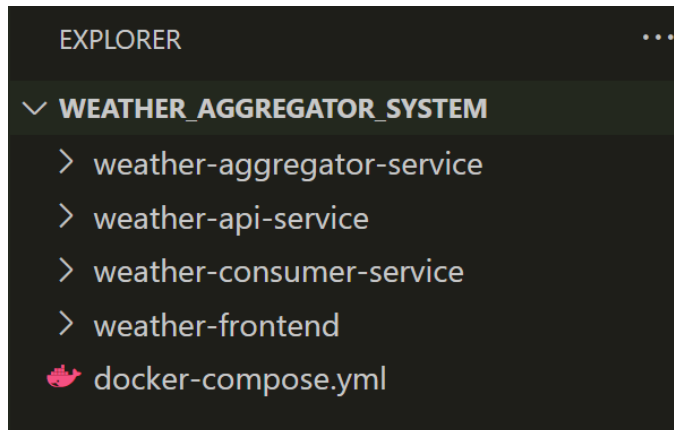
3. Получение ключа OpenWeatherMap API

aggregator_system

Active

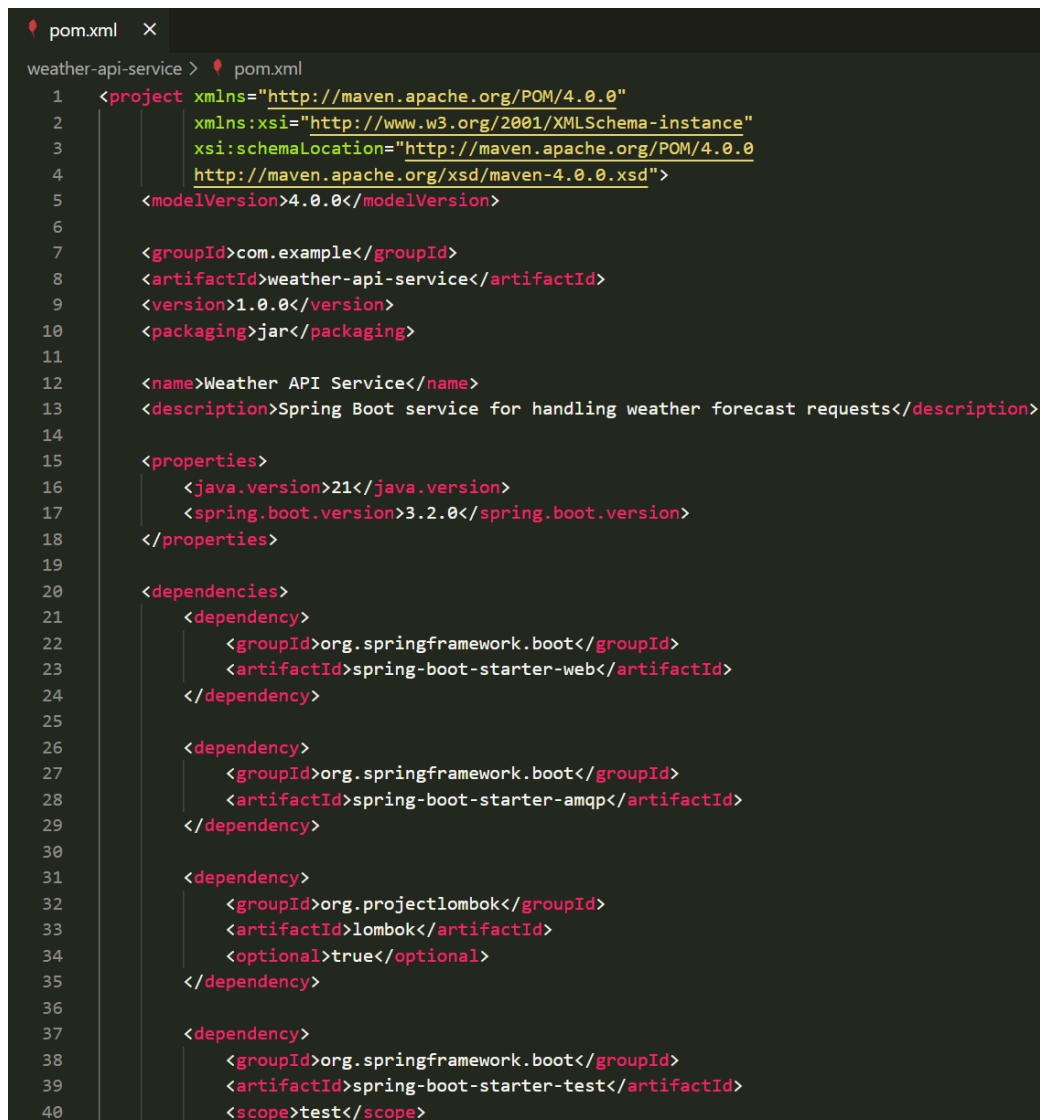
Шаг 2: Создание структуры проекта

4. Структура проекта



Шаг 3: Разработка weather-api-service

5. Создание pom.xml



```
41     </dependency>
42 </dependencies>
43
44 <build>
45     <plugins>
46         <plugin>
47             <groupId>org.springframework.boot</groupId>
48             <artifactId>spring-boot-maven-plugin</artifactId>
49         </plugin>
50     </plugins>
51 </build>
52
53 </project>
```

6. Создание application.yml

```
! application.yml X
weather-api-service > src > main > resources > ! application.yml
1  server:
2    port: 8080
3
4  spring:
5    application:
6      name: weather-api-service
7    rabbitmq:
8      host: localhost
9      port: 5672
10     username: admin
11     password: admin
12
13  rabbitmq:
14    queue:
15      request: weather.request.queue
16      aggregated: weather.aggregated.queue
17    exchange:
18      weather: weather.exchange
19    routing-key:
20      request: weather.request
21      aggregated: weather.aggregated
```

7. Создание основного класса приложения

```
J WeatherApiServiceApplication.java 1 X
weather-api-service > src > main > java > com > example > weatherapi > J WeatherApiServiceApplication.java > WeatherApiServiceApplication.java
1  package com.example.weatherapi;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6  @SpringBootApplication
7  public class WeatherApiServiceApplication {
8      Run | Debug
9      public static void main(String[] args) {
10         SpringApplication.run(WeatherApiServiceApplication.class, args);
11     }
12 }
```

8. Конфигурация RabbitMQ

```
J RabbitMQConfig.java 1 X
weather-api-service > src > main > java > com > example > weatherapi > config > J RabbitMQConfig.java > RabbitMQConfig
1  package com.example.weatherapi.config;
2
3  import java.util.Queue;
4
5  import org.springframework.amqp.core.*;
6  import org.springframework.context.annotation.Bean;
7  import org.springframework.context.annotation.Configuration;
8
9  @Configuration
10 public class RabbitMQConfig {
11
12     @Bean
13     public TopicExchange weatherExchange() {
14         return new TopicExchange("weather.exchange");
15     }
16
17     @Bean
18     public Queue requestQueue() {
19         return new Queue("weather.request.queue");
20     }
21
22     @Bean
23     public Queue aggregatedQueue() {
24         return new Queue("weather.aggregated.queue");
25     }
26
27     @Bean
28     public Binding requestBinding(Queue requestQueue, TopicExchange exchange) {
29         return BindingBuilder.bind(requestQueue).to(exchange).with("weather.request");
30     }
31
32     @Bean
33     public Binding aggregatedBinding(Queue aggregatedQueue, TopicExchange exchange) {
34         return BindingBuilder.bind(aggregatedQueue).to(exchange).with("weather.aggregated");
35     }
36 }
```

9. DTO-классы

```
J WeatherRequestDto.java 1 X
r-api-service > src > main > java > com > example > weatherapi > dto > J WeatherRequestDto.java >
1  package com.example.weatherapi.dto;
2
3  import lombok.Data;
4  import java.util.List;
5
6  @Data
7  public class WeatherRequestDto{
8      private List<String> cities;
9  }
```

```
J WeatherMessage.java 1 X
ther-api-service > src > main > java > com > example > weatherapi > dto > J WeatherMessage.java >
1  package com.example.weatherapi.dto;
2
3  import lombok.AllArgsConstructor;
4  import lombok.Data;
5  import lombok.NoArgsConstructor;
6
7  @Data
8  @AllArgsConstructor
9  @NoArgsConstructor
10 public class WeatherMessage {
11     private String city;
12     private String correlationId;
13 }
```

```
J AggregatedWeatherReport.java 1 X
service > src > main > java > com > example > weatherapi > dto > J AggregatedWeatherReport.java >
1  package com.example.weatherapi.dto;
2
3  import lombok.AllArgsConstructor;
4  import lombok.Data;
5  import lombok.NoArgsConstructor;
6  import java.util.Map;
7
8  @Data
9  @AllArgsConstructor
10 @NoArgsConstructor
11
12 public class AggregatedWeatherReport {
13     private String correlationId;
14     private Map<String, Object> weatherData;
15 }
```

10. Сервис и контроллер

Сервис:

```
J WeatherService.java 1 X
weather-api-service > src > main > java > com > example > weatherapi > J WeatherService.java > WeatherService
1 package com.example.weatherapi.service;
2
3 import com.example.weatherapi.dto.*;
4 import org.springframework.amqp.rabbit.annotation.RabbitListener;
5 import org.springframework.amqp.rabbit.core.RabbitTemplate;
6 import org.springframework.stereotype.Service;
7
8 import java.util.*;
9 import java.util.concurrent.*;
10
11 @Service
12 public class WeatherService {
13     private final RabbitTemplate rabbitTemplate;
14     private final Map<String, CompletableFuture<AggregatedWeatherReport>> pendingRequests = new ConcurrentHashMap<>();
15
16     public WeatherService(RabbitTemplate rabbitTemplate) {
17         this.rabbitTemplate = rabbitTemplate;
18     }
19
20     public AggregatedWeatherReport processWeatherRequest(List<String> cities) throws Exception {
21         String correlationId = UUID.randomUUID().toString();
22         CompletableFuture<AggregatedWeatherReport> future = new CompletableFuture<>();
23         pendingRequests.put(correlationId, future);
24
25         for (String city : cities) {
26             rabbitTemplate.convertAndSend("weather.exchange", "weather.request",
27                 new WeatherMessage(city, correlationId));
28         }
29
30         return future.get(30, TimeUnit.SECONDS);
31     }
32
33     @RabbitListener(queues = "weather.aggregated.queue")
34     public void receiveAggregatedReport(AggregatedWeatherReport report) {
35         CompletableFuture<AggregatedWeatherReport> future = pendingRequests.remove(report.getCorrelationId());
36         if (future != null) {
37             future.complete(report);
38         }
39     }
40 }
```

Контроллер:

```
J WeatherController.java 1 X
weather-api-service > src > main > java > com > example > weatherapi > J WeatherController.java > WeatherController
1 package com.example.weatherapi.controller;
2
3 import com.example.weatherapi.dto.WeatherRequestDto;
4 import com.example.weatherapi.dto.AggregatedWeatherReport;
5 import com.example.weatherapi.service.WeatherService;
6 import org.springframework.web.bind.annotation.*;
7
8 @RestController
9 @RequestMapping("/api/weather")
10 public class WeatherController {
11     private final WeatherService weatherService;
12
13     public WeatherController(WeatherService weatherService) {
14         this.weatherService = weatherService;
15     }
16
17     @PostMapping("/forecast")
18     public AggregatedWeatherReport getForecast(@RequestBody WeatherRequestDto request) throws Exception {
19         return weatherService.processWeatherRequest(request.getCities());
20     }
21 }
```

Шаг 4: Разработка weather-consumer-service

11. pom.xml

```

pom.xml x
weather-consumer-service > pom.xml
1  <project xmlns="http://maven.apache.org/POM/4.0.0"
2      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
4      http://maven.apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6
7      <groupId>com.example</groupId>
8      <artifactId>weather-consumer-service</artifactId>
9      <version>1.0.0</version>
10     <packaging>jar</packaging>
11
12     <name>Weather Consumer Service</name>
13     <description>Consumes weather requests and fetches data from OpenWeatherMap API</description>
14
15     <properties>
16         <java.version>21</java.version>
17         <spring.boot.version>3.2.0</spring.boot.version>
18     </properties>
19
20     <dependencies>
21         <dependency>
22             <groupId>org.springframework.boot</groupId>
23             <artifactId>spring-boot-starter-web</artifactId>
24         </dependency>
25
26         <dependency>
27             <groupId>org.springframework.boot</groupId>
28             <artifactId>spring-boot-starter-amqp</artifactId>
29         </dependency>
30
31         <dependency>
32             <groupId>com.fasterxml.jackson.core</groupId>
33             <artifactId>jackson-databind</artifactId>
34         </dependency>
35
36         <dependency>
37             <groupId>org.projectlombok</groupId>
38             <artifactId>lombok</artifactId>
39             <optional>true</optional>
40         </dependency>
41
42         <dependency>
43             <groupId>org.springframework.boot</groupId>
44             <artifactId>spring-boot-starter-test</artifactId>
45             <scope>test</scope>
46         </dependency>
47     </dependencies>
48
49     <build>
50         <plugins>
51             <plugin>
52                 <groupId>org.springframework.boot</groupId>
53                 <artifactId>spring-boot-maven-plugin</artifactId>
54             </plugin>
55         </plugins>
56     </build>
57
58 </project>

```


12. application.yml

```
! application.yml X
weather-consumer-service > src > main > resources > ! application.yml
1  server:
2    port: 8081
3
4  spring:
5    application:
6      name: weather-consumer-service
7    rabbitmq:
8      host: localhost
9      port: 5672
10     username: admin
11     password: admin
12
13  weather:
14    api:
15      key:
16      url: https://api.openweathermap.org/data/2.5/weather
17      delay: 1000
```

13. Основной класс приложения

```
J WeatherConsumerServiceApplication.java 2 X
: > main > java > com > example > weatherconsumer > J WeatherConsumerServiceApplication.java > WeatherConsumerServ
1  package com.example.weatherconsumer;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6  @SpringBootApplication
7  public class WeatherConsumerServiceApplication {
8      public static void main(String[] args) {
9          SpringApplication.run(WeatherConsumerServiceApplication.class, args);
10     }
11 }
```

14. DTO-классы

```
J WeatherMessage.java 2 X
umer-service > src > main > java > com > example > weatherconsumer > dto > J WeatherMessage.java
1  package com.example.weatherconsumer.dto;
2
3  import lombok.AllArgsConstructor;
4  import lombok.Data;
5  import lombok.NoArgsConstructor;
6
7  @Data
8  @AllArgsConstructor
9  @NoArgsConstructor
10 public class WeatherMessage {
11     private String city;
12     private String correlationId;
13 }
```

```
J WeatherResponse.java 1 X
h > java > com > example > weatherconsumer > dto > J WeatherResponse.java > WeatherResponse
1 package com.example.weatherconsumer.dto;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6
7 import java.util.Map;
8
9 @Data
10 @AllArgsConstructor
11 @NoArgsConstructor
12
13 public class WeatherResponse {
14     private String city;
15     private String correlationId;
16     private Map<String, Object> data;
17 }
```

```
J OpenWeatherMapResponse.java 1 X
ample > weatherconsumer > dto > J OpenWeatherMapResponse.java > OpenWeatherMapResponse
1 package com.example.weatherconsumer.dto;
2
3 import lombok.Data;
4 import java.util.Map;
5
6 @Data
7 public class OpenWeatherMapResponse {
8     private Map<String, Object> main;
9     private Map<String, Object> wind;
10     private String name;
11 }
```

15. Клиент для OpenWeatherMap API

```
J WeatherApiClient.java 2 X
weather-consumer-service > src > main > java > com > example > weatherconsumer > client > J WeatherApiClient.java > WeatherApiClient
1 package com.example.weatherconsumer.client;
2
3 import com.example.weatherconsumer.dto.OpenWeatherMapResponse;
4 import org.springframework.beans.factory.annotation.Value;
5 import org.springframework.stereotype.Component;
6 import org.springframework.web.client.RestTemplate;
7
8 @Component
9 public class WeatherApiClient {
10
11     @Value("${weather.api.key}")
12     private String apiKey;
13
14     @Value("${weather.api.url}")
15     private String apiUrl;
16
17     private final RestTemplate restTemplate = new RestTemplate();
18
19     public OpenWeatherMapResponse fetchWeather(String city) {
20         String url = String.format(format: "%s?q=%s&appid=%s&units=metric", apiUrl, city, apiKey);
21         return restTemplate.getForObject(url, OpenWeatherMapResponse.class);
22     }
23 }
```

16. Основная бизнес-логика

```
J WeatherConsumerService.java 1 X
weather-consumer-service > src > main > java > com > example > weatherconsumer > service > J WeatherConsumerService.java > WeatherConsumerService
1  package com.example.weatherconsumer.service;
2
3  import com.example.weatherconsumer.client.WeatherApiClient;
4  import com.example.weatherconsumer.dto.*;
5  import com.rabbitmq.client.Channel;
6  import org.springframework.amqp.core.Message;
7  import org.springframework.amqp.rabbit.annotation.RabbitListener;
8  import org.springframework.amqp.rabbit.core.RabbitTemplate;
9  import org.springframework.beans.factory.annotation.Value;
10 import org.springframework.stereotype.Service;
11
12 import java.util.HashMap;
13 import java.util.Map;
14
15 @Service
16 public class WeatherConsumerService {
17
18     private final WeatherApiClient apiClient;
19     private final RabbitTemplate rabbitTemplate;
20
21     @Value("${weather.api.delay}")
22     private Long delay;
23
24     public WeatherConsumerService(WeatherApiClient apiClient, RabbitTemplate rabbitTemplate) {
25         this.apiClient = apiClient;
26         this.rabbitTemplate = rabbitTemplate;
27     }
28
29     @RabbitListener(queues = "weather.request.queue", ackMode = "MANUAL")
30     public void consumeWeatherRequest(WeatherMessage request, Channel channel, Message message) throws Exception {
31         try {
32             Thread.sleep(delay);
33
34             OpenWeatherMapResponse response = apiClient.fetchWeather(request.getCity());
35
36             Map<String, Object> data = new HashMap<>();
37             data.put(key: "temp", response.getMain().get(key: "temp"));
38             data.put(key: "pressure", response.getMain().get(key: "pressure"));
39             data.put(key: "humidity", response.getMain().get(key: "humidity"));
40             data.put(key: "wind", response.getWind());
41             data.put(key: "city", response.getName());
42
43             WeatherResponse weatherResponse = new WeatherResponse(
44                 request.getCity(),
45                 request.getCorrelationId(),
46                 data
47             );
48
49             rabbitTemplate.convertAndSend("weather.exchange", "weather.response", weatherResponse);
50
51             channel.basicAck(message.getMessageProperties().getDeliveryTag(), false);
52         } catch (Exception e) {
53             e.printStackTrace();
54             channel.basicNack(message.getMessageProperties().getDeliveryTag(), false, true);
55         }
56     }
57 }
```

Шаг 5: Разработка weather-aggregator-service

17. application.yml

```
! application.yml X
weather-aggregator-service > src > main > resources > ! application.yml
1  server:
2    port: 8082
3
4  spring:
5    application:
6      name: weather-aggregator-service
7    rabbitmq:
8      host: localhost
9      port: 5672
10     username: admin
11     password: admin
12
13  aggregator:
14    timeout: 30
```

18. DTO-классы

```
J WeatherResponse.java X
service > src > main > java > com > example > weatheraggregator > dto > J WeatherResponse.java >
1  package com.example.weatheraggregator.dto;
2
3  import lombok.AllArgsConstructor;
4  import lombok.Data;
5  import lombok.NoArgsConstructor;
6
7  import java.util.Map;
8
9  @Data
10 @AllArgsConstructor
11 @NoArgsConstructor
12 public class WeatherResponse {
13     private String city;
14     private String correlationId;
15     private Map<String, Object> data;
16 }
```

```

J AggregatedWeatherReport.java 2 X
example > weatheraggregator > dto > J AggregatedWeatherReport.java > AggregatedWeatherReport
1  package com.example.weatheraggregator.dto;
2
3  import lombok.AllArgsConstructor;
4  import lombok.Data;
5  import lombok.NoArgsConstructor;
6
7  import java.util.List;
8  import java.util.Map;
9
10 @Data
11 @AllArgsConstructor
12 @NoArgsConstructor
13
14 public class AggregatedWeatherReport {
15     private String correlationId;
16     private List<Map<String, Object>> cityReports;
17 }

```

19. Класс сервиса

```

J WeatherAggregatorService.java 1 X
main > java > com > example > weatheraggregator > service > J WeatherAggregatorService.java > WeatherAggregatorService > AggregationContext
1  package com.example.weatheraggregator.service;
2
3  import com.example.weatheraggregator.dto.AggregatedWeatherReport;
4  import com.example.weatheraggregator.dto.WeatherResponse;
5  import org.springframework.amqp.rabbit.annotation.RabbitListener;
6  import org.springframework.amqp.rabbit.core.RabbitTemplate;
7  import org.springframework.beans.factory.annotation.Value;
8  import org.springframework.scheduling.annotation.Scheduled;
9  import org.springframework.stereotype.Service;
10
11 import java.time.Instant;
12 import java.util.*;
13 import java.util.concurrent.ConcurrentHashMap;
14
15 @Service
16 public class WeatherAggregatorService {
17
18     private final RabbitTemplate rabbitTemplate;
19     private final Map<String, AggregationContext> aggregationStore = new ConcurrentHashMap<>();
20
21     @Value("${aggregator.timeout}")
22     private Long aggregationTimeout;
23
24     public WeatherAggregatorService(RabbitTemplate rabbitTemplate) {
25         this.rabbitTemplate = rabbitTemplate;
26     }
27
28     private static class AggregationContext {
29         private final String correlationId;
30         private final int totalCities;
31         private final List<Map<String, Object>> cityReports = new ArrayList<>();
32         private int receivedCount = 0;
33         private final Instant startTime;
34
35         public AggregationContext(String correlationId, int totalCities) {
36             this.correlationId = correlationId;
37             this.totalCities = totalCities;
38             this.startTime = Instant.now();
39         }
40     }

```

```

41     public synchronized void addResponse(Map<String, Object> data) {
42         cityReports.add(data);
43         receivedCount++;
44     }
45
46     public boolean isComplete() {
47         return receivedCount >= totalCities;
48     }
49
50     public boolean isExpired(Long timeoutSeconds) {
51         return Instant.now().minusSeconds(timeoutSeconds).isAfter(startTime);
52     }
53
54     public AggregatedWeatherReport toReport() {
55         return new AggregatedWeatherReport(correlationId, cityReports);
56     }
57 }
58
59 @RabbitListener(queues = "weather.response.queue")
60 public void aggregateWeatherResponse(WeatherResponse response) {
61     String correlationId = response.getCorrelationId();
62
63     int totalCities = 3;
64
65     AggregationContext context = aggregationStore.computeIfAbsent(
66         correlationId,
67         id -> new AggregationContext(correlationId, totalCities)
68     );
69
70     context.addResponse(response.getData());
71
72     if (context.isComplete()) {
73         AggregatedWeatherReport report = context.toReport();
74
75         rabbitTemplate.convertAndSend("weather.exchange", "weather.aggregated", report);
76         aggregationStore.remove(correlationId);
77
78         System.out.println("Aggregated report sent for correlationId: " + correlationId);
79     }
80 }
81
82 @Scheduled(fixedRate = 10000)
83 public void cleanupExpiredAggregations() {
84     aggregationStore.entrySet().removeIf(entry -> {
85         AggregationContext ctx = entry.getValue();
86         if (ctx.isExpired(aggregationTimeout)) {
87             System.out.println("Aggregation expired and removed: " + ctx.correlationId);
88             return true;
89         }
90         return false;
91     });
92 }
93
94 }

```

20. Основной класс приложения

```
WeatherAggregatorServiceApplication.java 2 X
src > main > java > com > example > weatheraggregator > WeatherAggregatorServiceApplication.java > WeatherAg

1  package com.example.weatheraggregator;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5  import org.springframework.scheduling.annotation.EnableScheduling;
6
7  @SpringBootApplication
8  @EnableScheduling
9  public class WeatherAggregatorServiceApplication {
10     Run | Debug
11     public static void main(String[] args) {
12         SpringApplication.run(WeatherAggregatorServiceApplication.class, args);
13     }
14 }
```

Шаг 6: Разработка weather-frontend

21. index.html

```
index.html X
weather-frontend > index.html > html

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Weather Forecast Aggregator</title>
7      <link rel="stylesheet" href="style.css">
8  </head>
9  <body>
10     <div class="container">
11         <h1>Weather Forecast Aggregator</h1>
12
13         <textarea id="citiesInput" placeholder="Enter city names separated by commas (e.g., London, Paris, Berlin)"></textarea>
14         <button id="getForecastBtn">Get Forecast</button>
15
16         <div id="loading" class="hidden">Loading...</div>
17
18         <table id="resultsTable" class="hidden">
19             <thead>
20                 <tr>
21                     <th>City</th>
22                     <th>Temperature (°C)</th>
23                     <th>Weather</th>
24                     <th>Humidity (%)</th>
25                 </tr>
26             </thead>
27             <tbody id="resultsBody"></tbody>
28         </table>
29
30         <div id="error" class="hidden"></div>
31     </div>
32     <script src="app.js"></script>
33 </body>
34 </html>
```

22. style.css

```
# style.css ×
weather-frontend > # style.css > button: hover
1  body {
2      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
3      background: #f0f4f8;
4      display: flex;
5      justify-content: center;
6      align-items: flex-start;
7      height: 100vh;
8      margin: 0;
9      padding-top: 50px;
10 }
11
12 .container {
13     width: 600px;
14     background: white;
15     border-radius: 12px;
16     padding: 20px 30px;
17     box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
18 }
19
20 h1 {
21     text-align: center;
22     color: #2b6777;
23 }
24
25 textarea {
26     width: 100%;
27     height: 80px;
28     padding: 10px;
29     border: 1px solid #ccc;
30     border-radius: 8px;
31     resize: none;
32 }
33
34 button {
35     width: 100%;
36     padding: 10px;
37     background-color: #2b6777;
38     color: white;
39     border: none;
40     border-radius: 8px;
41     margin-top: 10px;
42     cursor: pointer;
43     font-size: 16px;
44 }
45
```



```
46 button:hover {
47   background-color: #3a8891;
48 }
49
50 #loading {
51   text-align: center;
52   color: #2b6777;
53   margin-top: 15px;
54   font-size: 18px;
55 }
56
57 .hidden {
58   display: none;
59 }
60
61 #error {
62   color: red;
63   margin-top: 10px;
64   text-align: center;
65 }
66
67 table {
68   width: 100%;
69   border-collapse: collapse;
70   margin-top: 20px;
71 }
72
73 th, td {
74   border-bottom: 1px solid #ddd;
75   padding: 8px;
76   text-align: center;
77 }
78
79 th {
80   background-color: #2b6777;
81   color: white;
82 }
```

23. app.js

```
JS app.js X
weather-frontend > JS app.js > ...
1 document.getElementById('getForecastBtn').addEventListener('click', async () => {
2   const input = document.getElementById('citiesInput').value.trim();
3   const loading = document.getElementById('loading');
4   const table = document.getElementById('resultsTable');
5   const tbody = document.getElementById('resultsBody');
6   const error = document.getElementById('error');
7
8   if (!input) {
9     alert("Please enter at least one city name!");
10    return;
11  }
12
13  const cities = input.split(',').map(c => c.trim()).filter(c => c);
14
15  loading.classList.remove('hidden');
16  table.classList.add('hidden');
17  error.classList.add('hidden');
18
19  try {
20    const response = await fetch('http://localhost:8080/api/weather/forecast', {
21      method: 'POST',
22      headers: {'Content-Type': 'application/json'},
23      body: JSON.stringify({ cities })
24    });
25
26    if (!response.ok) {
27      throw new Error(`Server error: ${response.status}`);
28    }
29
30    const data = await response.json();
31
32    tbody.innerHTML = '';
33
34    data.cityReports.forEach(report => {
```

```
35     const city = report.name || report.city || 'Unknown';
36     const temp = report.main?.temp ?? '-';
37     const weather = report.weather?.[0]?.description ?? '-';
38     const humidity = report.main?.humidity ?? '-';
39
40     const row = `<tr>
41         <td>${city}</td>
42         <td>${temp}</td>
43         <td>${weather}</td>
44         <td>${humidity}</td>
45     </tr>`;
46
47     tbody.insertAdjacentHTML('beforeend', row);
48 });
49
50 loading.classList.add('hidden');
51 table.classList.remove('hidden');
52 } catch (err) {
53     loading.classList.add('hidden');
54     error.textContent = `${err.message}`;
55     error.classList.remove('hidden');
56 }
57 });
```

Шаг 7: Сборка и запуск

24. Сборка

cd C:\Users\Admin\Documents\weather_aggregator_system\weather-api-service
mvn clean package -DskipTests

```
[INFO] Replacing main artifact C:\Users\Admin\Documents\weather_aggregator_system\weather-api-service\target\weather-api-service-1.0.0.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\Users\Admin\Documents\weather_aggregator_system\weather-api-service\target\weather-api-service-1.0.0.jar.original
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 23.998 s
[INFO] Finished at: 2025-11-02T17:40:55+03:00
[INFO] -----
```

cd C:\Users\Admin\Documents\weather_aggregator_system\weather-consumer-service
mvn clean package -DskipTests

```
[INFO] The original artifact has been renamed to C:\Users\Admin\Documents\weather_aggregator_system\weather-consumer-service\target\weather-consumer-service-1.0.0.jar.original
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.458 s
[INFO] Finished at: 2025-11-02T17:45:14+03:00
[INFO] -----
```

cd C:\Users\Admin\Documents\weather_aggregator_system\weather-aggregator-service
mvn clean package -DskipTests

```
[INFO] The original artifact has been renamed to C:\Users\Admin\Documents\weather_aggregator_system\weather-aggregator-service\target\weather-aggregator-service-1.0.0.jar.original
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 8.964 s
[INFO] Finished at: 2025-11-02T17:47:02+03:00
[INFO] -----
```


27. Добавление CorsConfig

```
J CorsConfig.java 1 X
weather-api-service > src > main > java > com > example > weatherapi > J CorsConfig.java > CorsConfig
1  package com.example.weatherapi;
2
3  import org.springframework.context.annotation.Bean;
4  import org.springframework.context.annotation.Configuration;
5  import org.springframework.web.cors.CorsConfiguration;
6  import org.springframework.web.cors.CorsConfigurationSource;
7  import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
8  import org.springframework.web.filter.CorsFilter;
9
10 @Configuration
11 public class CorsConfig {
12
13     @Bean
14     public CorsFilter corsFilter() {
15         CorsConfiguration config = new CorsConfiguration();
16         config.setAllowCredentials(allowCredentials: true);
17         config.addAllowedOrigin(origin: "http://localhost:8085");
18         config.addAllowedOrigin(origin: "http://127.0.0.1:8085");
19         config.addAllowedHeader(allowedHeader: "*");
20         config.addAllowedMethod(method: "*");
21
22         UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
23         source.registerCorsConfiguration(pattern: "/*", config);
24
25         return new CorsFilter(source);
26     }
27 }
```

28. Добавление RabbitConfig

```
J RabbitConfig.java X
weather-api-service > src > main > java > com > example > weatherapi > J RabbitConfig.java > RabbitConfig
1  package com.example.weatherapi;
2
3  import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
4  import org.springframework.amqp.support.converter.MessageConverter;
5  import org.springframework.context.annotation.Bean;
6  import org.springframework.context.annotation.Configuration;
7
8  @Configuration
9  public class RabbitConfig {
10
11     @Bean
12     public MessageConverter jMessageConverter() {
13         return new Jackson2JsonMessageConverter();
14     }
15 }
```

29. Обновление WeatherService

```
J WeatherService.java 1 X
weather-api-service > src > main > java > com > example > weatherapi > J WeatherService.java > WeatherService
1  package com.example.weatherapi.service;
2
3  import com.example.weatherapi.dto.*;
4  import org.springframework.amqp.rabbit.annotation.RabbitListener;
5  import org.springframework.amqp.rabbit.core.RabbitTemplate;
6  import org.springframework.stereotype.Service;
7
8  import java.util.*;
9  import java.util.concurrent.*;
10
11 @Service
12 public class WeatherService {
13     private final RabbitTemplate rabbitTemplate;
14     private final Map<String, CompletableFuture<AggregatedWeatherReport>> pendingRequests = new ConcurrentHashMap<>();
15
16     public WeatherService(RabbitTemplate rabbitTemplate) {
17         this.rabbitTemplate = rabbitTemplate;
18     }
19
20     public AggregatedWeatherReport processWeatherRequest(List<String> cities) throws Exception {
21         String correlationId = UUID.randomUUID().toString();
22         CompletableFuture<AggregatedWeatherReport> future = new CompletableFuture<>();
23         pendingRequests.put(correlationId, future);
24
25         for (String city : cities) {
26             rabbitTemplate.convertAndSend("weather.exchange", "weather.request",
27                 new WeatherMessage(city, correlationId));
28         }
29
30         System.out.println("Sent weather request for cities: " + cities + " with ID: " + correlationId);
31
32         return null;
33     }
34
35     @RabbitListener(queues = "weather.aggregated.queue")
36     public void receiveAggregatedReport(AggregatedWeatherReport report) {
37         CompletableFuture<AggregatedWeatherReport> future = pendingRequests.remove(report.getCorrelationId());
38         if (future != null) {
39             future.complete(report);
40         }
41     }
42 }
```

30. WeatherController

```

J WeatherController.java 2 X
weather-api-service > src > main > java > com > example > weatherapi > J WeatherController.java > WeatherController
1  package com.example.weatherapi.controller;
2
3  import com.example.weatherapi.dto.WeatherRequestDto;
4  import com.example.weatherapi.dto.AggregatedWeatherReport;
5  import com.example.weatherapi.service.WeatherService;
6  import org.springframework.web.bind.annotation.*;
7
8  import java.util.HashMap;
9  import java.util.Map;
10
11 @RestController
12 @RequestMapping("/api/weather")
13 public class WeatherController {
14     private final WeatherService weatherService;
15
16     public WeatherController(WeatherService weatherService) {
17         this.weatherService = weatherService;
18     }
19
20     @PostMapping("/forecast")
21     public Map<String, Object> getForecast(@RequestBody WeatherRequestDto request) {
22         try {
23             AggregatedWeatherReport report = weatherService.processWeatherRequest(request.getCities());
24
25             Map<String, Object> response = new HashMap<>();
26             response.put(key: "status", value: "success");
27             response.put(key: "message", value: "Weather request processed");
28
29             if (request.getCities() != null) {
30                 response.put(key: "cities", request.getCities());
31
32                 java.util.List<Map<String, Object>> testReports = new java.util.ArrayList<>();
33                 for (String city : request.getCities()) {
34                     Map<String, Object> testReport = new HashMap<>();
35                     testReport.put(key: "city", value: city);
36                     testReport.put(key: "temp", value: 15 + (int)(Math.random() * 10));
37                     testReport.put(key: "weather", value: "Sunny");
38                     testReport.put(key: "humidity", value: 65);
39                     testReports.add(testReport);
40                 }
41                 response.put(key: "cityReports", testReports);
42             } else {
43                 response.put(key: "cities", java.util.Collections.emptyList());
44                 response.put(key: "cityReports", java.util.Collections.emptyList());
45             }
46
47             return response;
48         } catch (Exception e) {
49             e.printStackTrace();
50             Map<String, Object> errorResponse = new HashMap<>();
51             errorResponse.put(key: "status", value: "error");
52             errorResponse.put(key: "message", value: e.getMessage());
53             return errorResponse;
54         }
55     }
56 }

```

31. Создание пути для контроллера

```

PROBLEMS 3 TERMINAL OUTPUT PORTS DEBUG CONSOLE
PS C:\Users\Admin\Documents\weather_aggregator_system> cd weather-api-service/PPS
PS C:\Users\Admin\Documents\weather_aggregator_system> cd weather-api-service/src/main/java/com/example/weatherapi
PS C:\Users\Admin\Documents\weather_aggregator_system\weather-api-service> cd src/main/java/com/example/weatherapi
PS C:\Users\Admin\Documents\weather_aggregator_system\weather-api-service> mkdir controller

Directory: C:\Users\Admin\Documents\weather_aggregator_system\weather-api-service\src\main\java\com\example\weatherapi

Mode                LastWriteTime         Length Name
----                -
d-----          02/11/2025   19:55             controller

PS C:\Users\Admin\Documents\weather_aggregator_system\weather-api-service\src\main\java\com\example\weatherapi> Move-Item WeatherController.java controller/
PS C:\Users\Admin\Documents\weather_aggregator_system\weather-api-service\src\main\java\com\example\weatherapi>

```

32. Обновления

```
WeatherService.java X
weather-api-service > src > main > java > com > example > weatherapi > service > WeatherService.java > WeatherService

1  package com.example.weatherapi.service;
2
3  import com.example.weatherapi.dto.*;
4  import org.springframework.amqp.rabbit.annotation.RabbitListener;
5  import org.springframework.amqp.rabbit.core.RabbitTemplate;
6  import org.springframework.stereotype.Service;
7
8  import java.util.*;
9  import java.util.concurrent.*;
10
11 @Service
12 public class WeatherService {
13     private final RabbitTemplate rabbitTemplate;
14     private final Map<String, CompletableFuture<AggregatedWeatherReport>> pendingRequests = new ConcurrentHashMap<>();
15
16     public WeatherService(RabbitTemplate rabbitTemplate) {
17         this.rabbitTemplate = rabbitTemplate;
18     }
19
20     public AggregatedWeatherReport processWeatherRequest(List<String> cities) throws Exception {
21         String correlationId = UUID.randomUUID().toString();
22         CompletableFuture<AggregatedWeatherReport> future = new CompletableFuture<>();
23         pendingRequests.put(correlationId, future);
24
25         for (String city : cities) {
26             rabbitTemplate.convertAndSend(exchange: "weather.exchange", routingKey: "weather.request",
27                 new WeatherMessage(city, correlationId));
28         }
29
30         System.out.println("Sent weather request for: " + cities);
31
32         return future.get(timeout: 15, TimeUnit.SECONDS);
33     }
34
35     @RabbitListener(queues = "weather.aggregated.queue")
36     public void receiveAggregatedReport(AggregatedWeatherReport report) {
37         CompletableFuture<AggregatedWeatherReport> future = pendingRequests.remove(report.getCorrelationId());
38         if (future != null) {
39             future.complete(report);
40         }
41     }
42 }
```

```
J WeatherController.java X
weather-api-service > src > main > java > com > example > weatherapi > controller > J WeatherController.java > WeatherController > getForecast

1  package com.example.weatherapi.controller;
2
3  import com.example.weatherapi.dto.WeatherRequestDto;
4  import com.example.weatherapi.dto.AggregatedWeatherReport;
5  import com.example.weatherapi.service.WeatherService;
6  import org.springframework.web.bind.annotation.*;
7
8  import java.util.HashMap;
9  import java.util.Map;
10
11  @RestController
12  @RequestMapping("/api/weather")
13  public class WeatherController {
14      private final WeatherService weatherService;
15
16      public WeatherController(WeatherService weatherService) {
17          this.weatherService = weatherService;
18      }
19
20      @PostMapping("/forecast")
21      public Map<String, Object> getForecast(@RequestBody WeatherRequestDto request) {
22          Map<String, Object> response = new HashMap<>();
23
24          try {
25              if (request.getCities() == null || request.getCities().isEmpty()) {
26                  response.put(key: "status", value: "error");
27                  response.put(key: "message", value: "City list cannot be empty");
28                  return response;
29              }
30
31              AggregatedWeatherReport report = weatherService.processWeatherRequest(request.getCities());
32
33              if (report == null || report.getCityReports() == null) {
34                  response.put(key: "status", value: "error");
35                  response.put(key: "message", value: "No data received from aggregator");
36                  return response;
37              }
38
39              response.put(key: "status", value: "success");
40              response.put(key: "message", value: "Weather data fetched successfully");
41              response.put(key: "cities", request.getCities());
42              response.put(key: "cityReports", report.getCityReports());
43              return response;
44          } catch (Exception e) {
45              e.printStackTrace();
46              response.put(key: "status", value: "error");
47              response.put(key: "message", e.getMessage());
48              return response;
49          }
50      }
51  }
52 }
```



```

J WeatherAggregatorService.java X
weather-aggregator-service > src > main > java > com > example > weatheraggregator > service > J WeatherAggregatorService.java > WeatherAggregatorS
1  package com.example.weatheraggregator.service;
2
3  import com.example.weatheraggregator.dto.AggregatedWeatherReport;
4  import com.example.weatheraggregator.dto.WeatherResponse;
5  import org.springframework.amqp.rabbit.annotation.RabbitListener;
6  import org.springframework.amqp.rabbit.core.RabbitTemplate;
7  import org.springframework.beans.factory.annotation.Value;
8  import org.springframework.scheduling.annotation.Scheduled;
9  import org.springframework.stereotype.Service;
10
11 import java.time.Instant;
12 import java.util.*;
13 import java.util.concurrent.ConcurrentHashMap;
14
15 @Service
16 public class WeatherAggregatorService {
17
18     private final RabbitTemplate rabbitTemplate;
19     private final Map<String, AggregationContext> aggregationStore = new ConcurrentHashMap<>();
20
21     @Value("${aggregator.timeout}")
22     private Long aggregationTimeout;
23
24     public WeatherAggregatorService(RabbitTemplate rabbitTemplate) {
25         this.rabbitTemplate = rabbitTemplate;
26     }
27
28     private static class AggregationContext {
29         private final String correlationId;
30         private final int totalCities;
31         private final List<Map<String, Object>> cityReports = new ArrayList<>();
32         private int receivedCount = 0;
33         private final Instant startTime;
34
35         public AggregationContext(String correlationId, int totalCities) {
36             this.correlationId = correlationId;
37             this.totalCities = totalCities;
38             this.startTime = Instant.now();
39         }
40
41         public synchronized void addResponse(Map<String, Object> data) {
42             cityReports.add(data);
43             receivedCount++;
44         }
45
46         public boolean isComplete() {
47             return receivedCount >= totalCities;
48         }
49
50         public boolean isExpired(long timeoutSeconds) {
51             return Instant.now().minusSeconds(timeoutSeconds).isAfter(startTime);
52         }
53
54         public AggregatedWeatherReport toReport() {
55             return new AggregatedWeatherReport(correlationId, cityReports);
56         }
57     }
58
59     @RabbitListener(queues = "weather.response.queue")
60     public void aggregateWeatherResponse(WeatherResponse response) {
61         String correlationId = response.getCorrelationId();
62
63         int totalCities = 3;
64
65         AggregationContext context = aggregationStore.computeIfAbsent(
66             correlationId,
67             id -> new AggregationContext(correlationId, totalCities)
68         );
69
70         context.addResponse(response.getData());
71
72         if (context.isComplete()) {
73             AggregatedWeatherReport report = context.toReport();
74
75             rabbitTemplate.convertAndSend(exchange: "weather.exchange", routingKey: "weather.aggregated", report);
76             aggregationStore.remove(correlationId);
77
78             System.out.println("Aggregated report sent for correlationId: " + correlationId);
79         }
80     }
81
82     @Scheduled(fixedRate = 10000)
83     public void cleanupExpiredAggregations() {
84         aggregationStore.entrySet().removeIf(entry -> {
85             AggregationContext ctx = entry.getValue();
86             if (ctx.isExpired(aggregationTimeout)) {
87                 System.out.println("Aggregation expired and removed: " + ctx.correlationId);
88                 return true;
89             }
90             return false;
91         });
92     }
93
94 }

```

Результат:

<http://localhost:8085/>

